Information Processing Letters 23 (1986) 159–162
North-Holland

22 October 1986

ABSENCE OF INDIVIDUAL STARVATION USING WEAK SEMAPHORES

Jan Tijmen UDDING

Institute for Biomedical Computing, Washington University, St. Louis, MO 63130, U.S.A.

Communicated by David Gries
Received 26 July 1985
Revised 2 August 1985

Keywords: Multi-programming, operating systems, analysis of algorithms

As far as scheduling is concerned, there are two types of semaphores: weak and strong. A semaphore is called *strong* if there is an upper bound on the number of times that a process may be prevented from passing that semaphore in favour of another process. A strong semaphore has a scheduling algorithm associated with it that guarantees absence of individual starvation at that semaphore. A semaphore is called *weak* if absence of individual starvation among processes blocked at that semaphore cannot be guaranteed. For weak semaphores, one assumption is made, however, viz. a process that executes a V-operation on a semaphore will not be the one to perform the next P-operation on that semaphore, if a process has been blocked at that semaphore. One of the waiting processes is allowed to pass the semaphore.

When solving mutual exclusion problems, one usually assumes the existence of strong semaphores. It is well known that strong semaphores can be implemented by weak ones [3,4]. However, the solutions presented in these papers, although rigorously proved, come out of the blue. In this paper we show that this problem can be solved in a systematic way. A solution can be derived by carefully massaging a partially correct, but obvious, program into a less obvious but correct one. Although it closely resembles the solution of Morris [4], its operation is quite different.

The following is the problem to be solved. A number of processes have a cyclic way of operating. In each cycle, a critical section (CS) and a noncritical section (NCS) occur. At any moment, at most one process is allowed to be in its critical section. The only synchronization primitives we have are weak semaphores. As usual we must guarantee, in addition to mutual exclusion, absence of deadlock and of individual starvation.

The standard solution [1], not guaranteeing absence of starvation in the case of weak semaphores, would be (*mutex* being a binary semaphore which is initially equal to 1)

do *true* → *NCS*; P(*mutex*); *CS*; V(*mutex*) **od**

We derive a program that guarantees absence of individual starvation as well by taking this program as a starting point. From now on, we consider only the critical section and its entrance and exit protocols.

The problem is that, in the presence of three or more processes, a process that performs a V(*mutex*) operation can arrive at P(*mutex*) again before all processes waiting at P(*mutex*) have passed. Hence, we have to prevent a process from arriving at P(*mutex*) after a V(*mutex*) operation until all processes waiting at P(*mutex*) have passed. A standard technique by now is to introduce for that purpose another semaphore *enter*, say, that together with *mutex* constitutes a split binary semaphore [2], which means that $0 \leqslant mutex + enter \leqslant 1$. Now, a number of processes assemble at P(*mutex*). Once the first of them is allowed to

pass this semaphore, *enter* should prohibit the arrival of following processes at P(*mutex*) until all processes assembled there have been sluiced through. As usual, we need a counter *nm*, say, for the number of processes waiting at P(*mutex*). Now the program looks like (*enter* being initially 1 and, hence, *mutex* being initially 0)

```
P(enter); nm := nm + 1;
          if B0 → V(mutex)
          □ B1 → V(enter)
          fi;
P(mutex); nm := nm − 1;
          CS;
          if nm > 0 → V(mutex)
          □ nm = 0 → V(enter)
          fi
```

From this program text it is clear that *mutex* + *enter* is a split binary semaphore and that *nm* is the number of processes waiting at P(*mutex*), provided that $mutex + enter = 1$. Moreover, we have retained mutual exclusion. Unfortunately, we have only moved the problem of individual starvation from *mutex* to *enter*, and B_0 and B_1 should be carefully chosen so as to guarantee absence of deadlock.

Let us focus our attention on B_0 and B_1 first. In order to avoid processes being indefinitely blocked at *mutex*, B_1 should be true only if there is at least one process waiting at P(*enter*), since $nm > 0$ when B_1 is passed. Therefore, we introduce the variable *ne* to denote the number of processes waiting at P(*enter*). Hence, we must have $B_1 \Rightarrow ne > 0$. On the other hand, there may be individual starvation at P(*enter*) if B_0 is allowed to become true while processes are waiting at P(*enter*). Hence, we must have $ne > 0 \Rightarrow \neg B_0$. In fact, we have no other choice than to make B_0 equal to $ne = 0$ and to make B_1 equal to $ne > 0$. Since *ne* should be modified by at most one process at a time, assignments to *ne* must be surrounded by the proper P- and V-operations. The program then becomes

```
P(X); ne := ne + 1; V(Y);
P(enter); nm := nm + 1; ne := ne − 1;
          if ne > 0 → V(enter)
          □ ne = 0 → V(mutex)
          fi;
P(mutex); nm := nm − 1;
          CS;
          if nm > 0 → V(mutex)
          □ nm = 0 → V(enter)
          fi
```

Trying not to introduce more semaphores than necessary, our first attempt is to use *mutex* and *enter* for X and Y. Then, implicitly, *ne* is modified by at most one process at a time, since these semaphores are binary. A precondition for the execution of V(Y) is $ne > 0$. When defining B_0 and B_1, we decided that a precondition for V(*mutex*) should be $ne = 0$ in order to avoid individual starvation at P(*enter*). The same argument holds here, and thus we should choose Y to be *enter*. Then it is apparent that we should not choose *mutex* for X, since the only process that could proceed in that case after a V(Y) operation is that process itself by performing the next P(*enter*) operation, which means that the statement list V(Y); P(*enter*) could be disposed of. Therefore, we choose for X the semaphore *enter* as well. With a few assertions added to ease the verification, the program developed thusfar looks like

```
P(enter) {ne ⩾ 0}; ne := ne + 1 {ne > 0};
                   V(enter);
P(enter) {ne > 0 ∧ nm ⩾ 0};
         nm := nm + 1; ne := ne − 1;
         if ne > 0 → {ne > 0} V(enter)
         □ ne = 0 → {ne = 0 ∧ nm > 0}
                    V(mutex)
         fi;
P(mutex) {ne = 0 ∧ nm > 0};
         nm := nm − 1;
         CS;
         if nm > 0 → {ne = 0 ∧ nm > 0}
                     V(mutex)
         □ nm = 0 → {ne = 0 ∧ nm = 0}
                     V(enter)
         fi
```

This program still guarantees mutual exclusion. Moreover, deadlock cannot occur since we have

$$enter = 1 \Rightarrow ne > 0 \vee (ne = 0 \wedge nm = 0)$$

and

$$mutex = 1 \Rightarrow nm > 0 \wedge ne = 0$$

as can easily be seen from the assertions. The only place with danger of individual starvation is the first P(*enter*). The second P(*enter*) is free from individual starvation and that is what we will now exploit to guarantee absence of individual starvation at the first P(*enter*) as well.

The key to guaranteeing absence of individual starvation at the first P(*enter*) is that passing that semaphore should have a higher priority than passing the second P(*enter*), for which we have already guaranteed absence of individual starvation. Hence, if a V(*enter*) operation is performed, we would like the first P(*enter*) to be passed rather than the second one. In fact, because of absence of individual starvation at the second P(*enter*), it is sufficient to guarantee that the last process blocked at its second P(*enter*) passes that semaphore only if no processes are waiting at their first P(*enter*). Hence, only if $ne = 1$ do we have to be careful.

If a process executes its third V(*enter*) operation, we have $ne = 0$ as we have seen before. If a process executes its first V(*enter*) operation and a process is waiting at its second P(*enter*), then $ne > 1$ and we do not care which of the two P(*enter*)'s is performed.

The only problem left is the second V(*enter*). If that operation is performed, processes waiting at the first P(*enter*) should have priority. They have that priority if no other process is waiting at its second P(*enter*) when a process executes its second V(*enter*), due to the assumption we made about weak semaphores that one of the processes blocked at a semaphore will pass that semaphore when a corresponding V-operation is performed. This means that when a process passes its second P(*enter*), no other process should be waiting there and no process should arrive there until this process has performed its second V(*enter*). This is, surprisingly enough, just a mutual exclusion problem. We introduce an additional binary semaphore to queue the processes blocked at their second P(*enter*). The final program now is (*queue* being initially 1)

```
P(enter); ne := ne + 1; V(enter);
P(queue); P(enter); nm := nm + 1;
                    ne := ne - 1;
                    if ne > 0 → V(enter)
                    □ ne = 0 → V(mutex)
                    fi;
V(queue);
P(mutex); nm := nm - 1;
          CS;
          if nm > 0 → V(mutex)
          □ nm = 0 → V(enter)
          fi
```

The introduction of *queue* does not lead to deadlock. The fact that no process is blocked at its second P(*enter*), when a process has passed its second P(*enter*) but not yet its subsequent V-operation, makes no difference since *enter* is a binary semaphore and there are no other P-operations between P(*queue*) and V(*queue*). There is no individual starvation at P(*queue*) either, since *ne* now simply denotes the number of processes blocked at P(*queue*) or their second P(*enter*).

Concluding remarks

A program has been derived that shows that strong semaphores can be implemented by weak ones. The techniques used for the derivation are standard, and it is straightforward, with the arguments used for the derivation, to deduce a formal correctness proof, which for the purpose of this paper is considered superfluous. In this program as in previous programs that implement strong semaphores, the order of two V-operations, viz. V(*enter*) and V(*queue*), turns out to be crucial. Here, however, the reason is clear. It just stems from a mutual exclusion problem.

Acknowledgment

I gratefully acknowledge the Eindhoven Tuesday Afternoon Club, in particular Nettie van Gasteren, and Jan L.A. van de Snepscheut for their valuable criticism on earlier drafts of this paper. Comments of David I. Burstyn and an anonymous referee led to quite some improvement of the presentation.

References

[1] E.W. Dijkstra, Cooperating sequential processes, EWD 123, Eindhoven Univ. of Technology, 1965; reprinted in: F. Genuys, ed., Programming Languages (Academic Press, New York, 1968).

[2] C.A.R. Hoare, Monitors: An operating system structuring concept, Comm. ACM 17 (10) (1974) 549–557; see also: Erratum, Comm. ACM 18 (2) (1975) 95.

[3] A.J. Martin and J.R. Burch, Fair mutual exclusion with unfair P and V operations, Inform. Process. Lett. 21 (2) (1985) 97–100.

[4] J.M. Morris, A starvation-free solution to the mutual exclusion problem, Inform. Process. Lett. 8 (2) (1979) 76–80.